

Importing libraries

Imports necessary libraries for the project, including pandas, numpy, scikit-learn modules, and visualization libraries like matplotlib and seaborn.

```
In [23]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

Loading data

Loads the dataset (Crop_recommendation.csv) using pandas. The path to the file is specified

```
In [24]: df=pd.read_csv('/content/Crop_recommendation.csv')
df
```

```
Out[24]:
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice
...	...	...	...	...	...	...	...	...
2195	107	34	32	26.774637	66.413269	6.780064	177.774507	coffee
2196	99	15	27	27.417112	56.636362	6.086922	127.924610	coffee
2197	118	33	30	24.131797	67.225123	6.362608	173.322839	coffee
2198	117	32	34	26.272418	52.127394	6.758793	127.175293	coffee
2199	104	18	30	23.603016	60.396475	6.779833	140.937041	coffee

2200 rows × 8 columns

The dataset above allow the users to build a predictive model to recommend the most suitable crops to grow in a particular farm based on various parameters.

Data fields:

- N - ratio of Nitrogen content in soil
- P - ratio of Phosphorous content in soil
- K - ratio of Potassium content in soil
- temperature - temperature in degree Celsius
- humidity - relative humidity in %
- ph - ph value of the soil
- rainfall - rainfall in mm

Prints the shape of the dataset, indicating the number of rows and columns.

```
In [25]: print("Shape of the Dataset :",df.shape)
```

```
Shape of the Dataset : (2200, 8)
```

Lets Checks for any missing values in the dataset.

```
In [26]: df.isna().sum()
```

```
Out[26]: N          0
P          0
K          0
temperature  0
humidity     0
ph           0
rainfall     0
label        0
dtype: int64
```

Prints the data types of each column in the dataset.

```
In [27]: df.dtypes
```

```
Out[27]: N          int64
P          int64
K          int64
temperature  float64
humidity     float64
ph           float64
rainfall     float64
label        object
dtype: object
```

Prints the count of unique values in the 'label' column, representing the different crops.

```
In [28]: df['label'].value_counts()
```

```
Out[28]: rice      100
maize    100
jute     100
cotton   100
coconut  100
papaya   100
orange   100
apple    100
muskmelon 100
watermelon 100
grapes   100
mango    100
banana   100
pomegranate 100
lentil   100
blackgram 100
mungbean 100
mothbeans 100
pigeonpeas 100
kidneybeans 100
chickpea 100
coffee  100
Name: label, dtype: int64
```

Exploratory Data Analysis

Groups the dataset by crop label and calculates the mean values for each feature.

```
In [29]: grouped = df.groupby(by='label').mean().reset_index()
grouped
```

Out[29]:

	label	N	P	K	temperature	humidity	ph	rainfall
0	apple	20.80	134.22	199.89	22.630942	92.333383	5.929663	112.654779
1	banana	100.23	82.01	50.05	27.376798	80.358123	5.983893	104.626980
2	blackgram	40.02	67.47	19.24	29.973340	65.118426	7.133952	67.884151
3	chickpea	40.09	67.79	79.92	18.872847	16.860439	7.336957	80.058977
4	coconut	21.98	16.93	30.59	27.409892	94.844272	5.976562	175.686646
5	coffee	101.20	28.74	29.94	25.540477	58.869846	6.790308	158.066295
6	cotton	117.77	46.24	19.56	23.988958	79.843474	6.912675	80.398043
7	grapes	23.18	132.53	200.11	23.849575	81.875228	6.025937	69.611829
8	jute	78.40	46.86	39.99	24.958376	79.639864	6.732778	174.792798
9	kidneybeans	20.75	67.54	20.05	20.115085	21.605357	5.749411	105.919778
10	lentil	18.77	68.36	19.41	24.509052	64.804785	6.927932	45.680454
11	maize	77.76	48.44	19.79	22.389204	65.092249	6.245190	84.766988
12	mango	20.07	27.18	29.92	31.208770	50.156573	5.766373	94.704515
13	mothbeans	21.44	48.01	20.23	28.194920	53.160418	6.831174	51.198487
14	mungbean	20.99	47.28	19.87	28.525775	85.499975	6.723957	48.403601
15	muskmelon	100.32	17.72	50.08	28.663066	92.342802	6.358805	24.689952
16	orange	19.58	16.55	10.01	22.765725	92.170209	7.016957	110.474969
17	papaya	49.88	59.05	50.04	33.723859	92.403388	6.741442	142.627839
18	pigeonpeas	20.73	67.73	20.29	27.741762	48.061633	5.794175	149.457564
19	pomegranate	18.87	18.75	40.21	21.837842	90.125504	6.429172	107.528442
20	rice	79.89	47.58	39.87	23.689332	82.272822	6.425471	236.181114
21	watermelon	99.42	17.00	50.22	25.591767	85.160375	6.495778	50.786219

Prints the top 5 crops requiring the most of each feature.

```
In [30]: print(f'*****')
for i in grouped.columns[1:]:
    print(f'Top 5 Most {i} requiring crops:')
    print(f'*****')
    for j ,k in grouped.sort_values(by=i,ascending=False)[:5][['label',i]].values:
        print(f'{j} --> {k}')
    print(f'*****')
```

```

*****
Top 5 Most N requiring crops:
*****

cotton --> 117.77
coffee --> 101.2
muskmelon --> 100.32
banana --> 100.23
watermelon --> 99.42
*****

Top 5 Most P requiring crops:
*****

apple --> 134.22
grapes --> 132.53
banana --> 82.01
lentil --> 68.36
chickpea --> 67.79
*****

Top 5 Most K requiring crops:
*****

grapes --> 200.11
apple --> 199.89
chickpea --> 79.92
watermelon --> 50.22
muskmelon --> 50.08
*****

Top 5 Most temperature requiring crops:
*****

papaya --> 33.7238587388
mango --> 31.2087701513
blackgram --> 29.9733396789
muskmelon --> 28.663065756
mungbean --> 28.5257747353
*****

Top 5 Most humidity requiring crops:
*****

coconut --> 94.84427180610001
papaya --> 92.4033876826
muskmelon --> 92.34280196089999
apple --> 92.3333828756
orange --> 92.17020876340001
*****

Top 5 Most ph requiring crops:
*****

chickpea --> 7.33695662374
blackgram --> 7.13395162948
orange --> 7.01695745276
lentil --> 6.927931571609999
cotton --> 6.91267549578
*****

Top 5 Most rainfall requiring crops:
*****

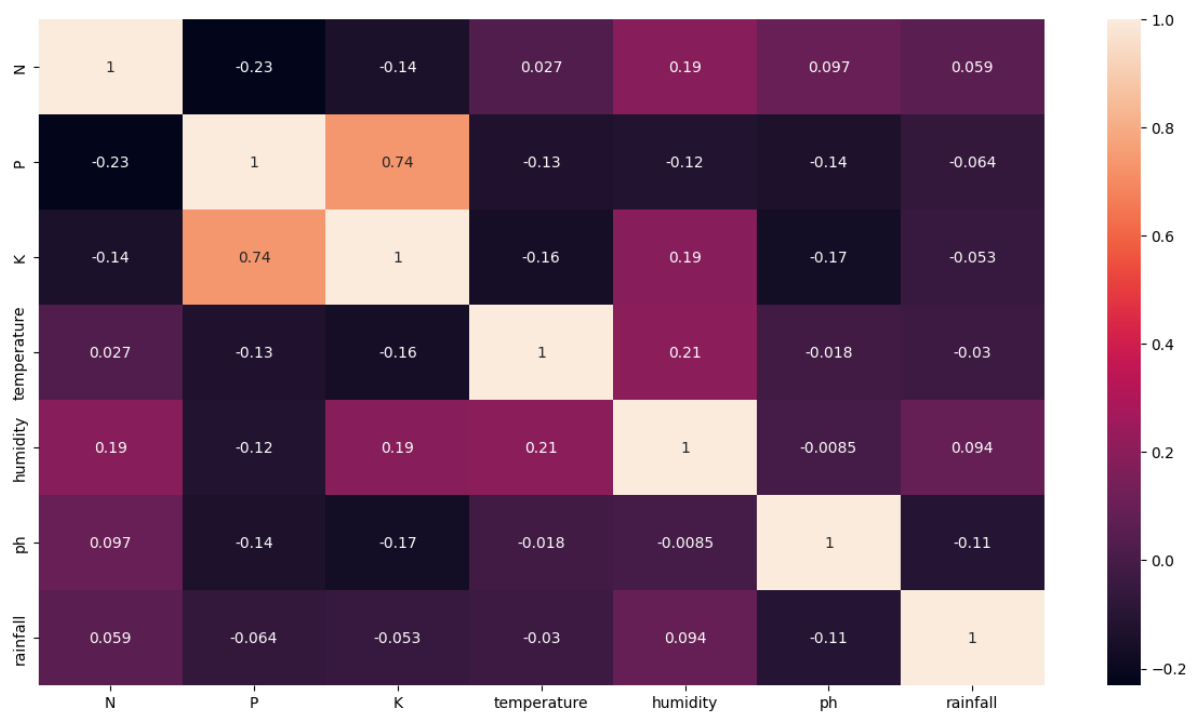
rice --> 236.181113594
coconut --> 175.686645804
jute --> 174.792797536
coffee --> 158.066294882
pigeonpeas --> 149.4575638135
*****

```

Plots a heatmap to visualize the correlation matrix of the dataset.

```
In [31]: sns.heatmap(df.corr(),annot=True)
```

```
Out[31]: <Axes: >
```



Creates a pair plot to explore relationships between different features in the dataset, with different colors for each crop.

```
In [32]: sns.pairplot(df,hue = 'label')
```

Output hidden; open in <https://colab.research.google.com> to view.

Creates bar plots to visualize the impact of different conditions on crops, including Nitrogen, Phosphorous, Potassium, Temperature, Humidity, pH, and Rainfall.

```
In [33]: plt.figure(layout='constrained')
plt.rcParams['figure.figsize'] = (15, 8)

plt.subplot(2, 4, 1)
sns.barplot(x='N', y='label', data=df)
plt.xlabel('Ratio of Nitrogen', fontsize=10)
plt.yticks(fontsize=10)

plt.subplot(2, 4, 2)
sns.barplot(x='P', y='label', data=df)
plt.xlabel('Ratio of Phosphorous', fontsize=10)
plt.yticks(fontsize=10)

plt.subplot(2, 4, 3)
sns.barplot(x='K', y='label', data=df)
plt.xlabel('Ratio of Potassium', fontsize=10)
plt.yticks(fontsize=10)

plt.subplot(2, 4, 4)
sns.barplot(x='temperature', y='label', data=df)
plt.xlabel('Temperature', fontsize=10)
plt.yticks(fontsize=10)

plt.subplot(2, 4, 5)
sns.barplot(x='humidity', y='label', data=df)
plt.xlabel('Humidity', fontsize=10)
plt.yticks(fontsize=10)

plt.subplot(2, 4, 6)
```

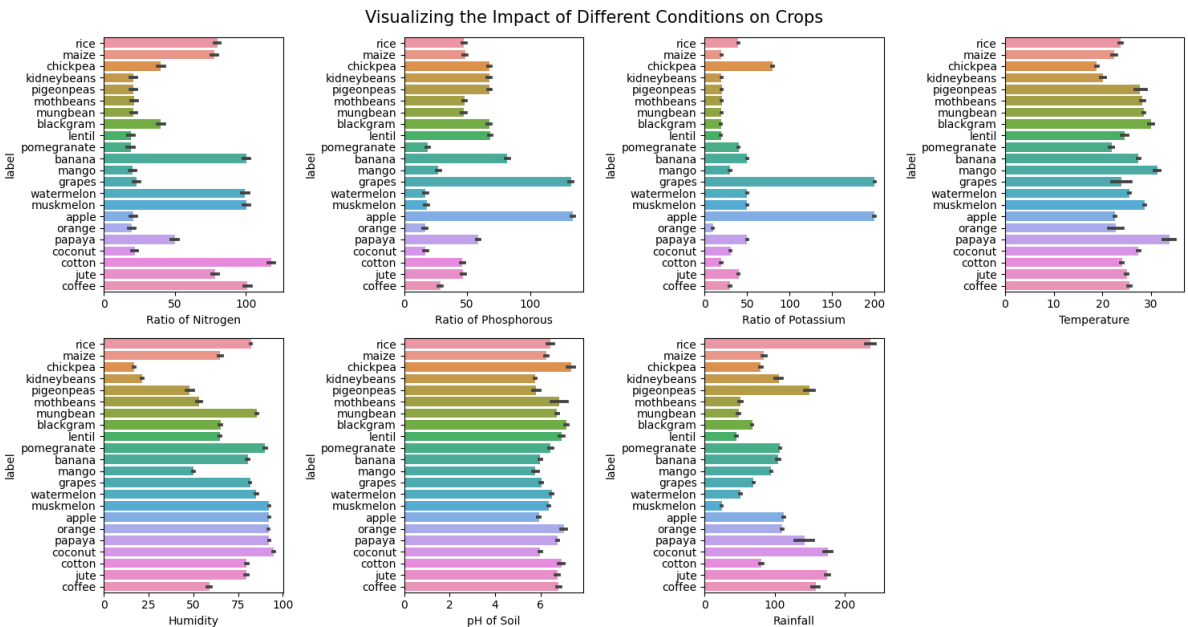
```

sns.barplot(x='ph', y='label', data=df)
plt.xlabel('pH of Soil', fontsize=10)
plt.yticks(fontsize=10)

plt.subplot(2, 4, 7)
sns.barplot(x='rainfall', y='label', data=df)
plt.xlabel('Rainfall', fontsize=10)
plt.yticks(fontsize=10)

plt.suptitle('Visualizing the Impact of Different Conditions on Crops', fontsize=15)
plt.tight_layout()
plt.show()

```

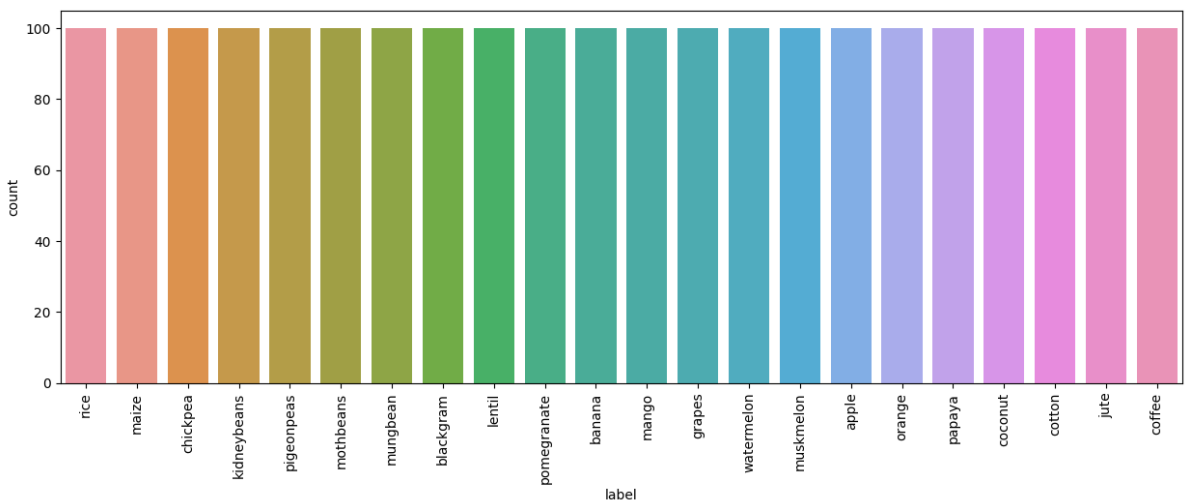


Plots a count plot to show the distribution of different crops.

```

In [34]: plt.figure(figsize=(15, 5))
sns.countplot(x='label', data=df)
plt.xticks(rotation=90)
plt.show()

```



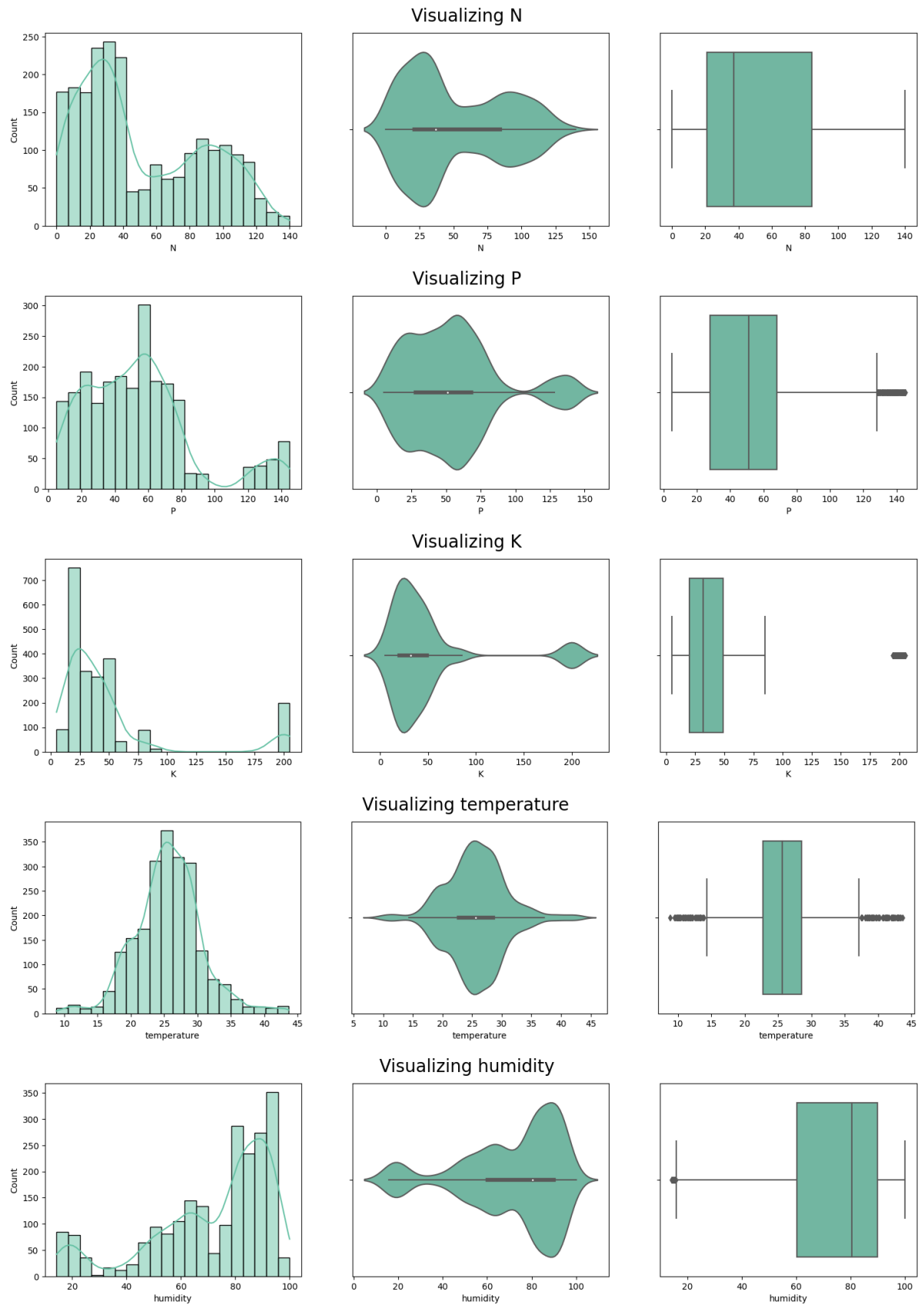
Uses histograms, violin plots, and boxplots to visualize the distribution of each feature.

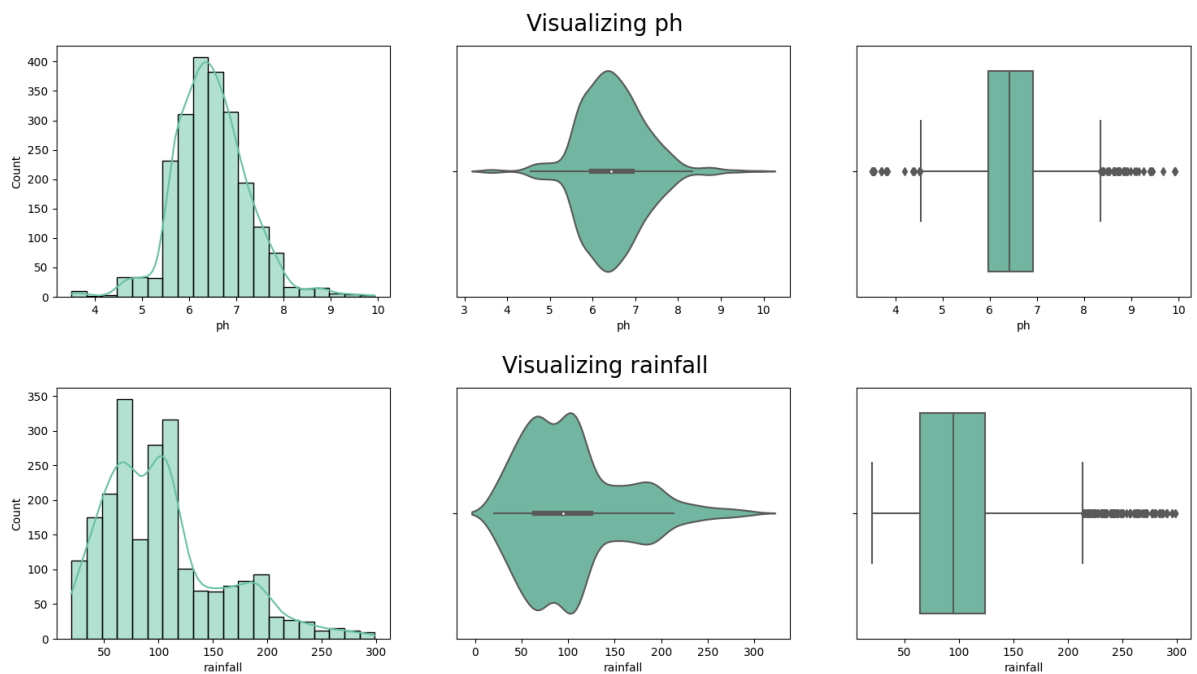
```

In [35]: plt.style.use('fast')
sns.set_palette("Set2")
for i in df.columns[:-1]:
    fig,ax=plt.subplots(1,3,figsize=(18,4))

```

```
sns.histplot(data=df,x=i,kde=True,bins=20,ax=ax[0])
sns.violinplot(data=df,x=i,ax=ax[1])
sns.boxplot(data=df,x=i,ax=ax[2])
plt.suptitle(f'Visualizing {i}',size=20)
```





Seperating features and target label

Prints the unique crop labels in the 'label' column.

```
In [36]: df['label'].unique()
```

```
Out[36]: array(['rice', 'maize', 'chickpea', 'kidneybeans', 'pigeonpeas',
        'mothbeans', 'mungbean', 'blackgram', 'lentil', 'pomegranate',
        'banana', 'mango', 'grapes', 'watermelon', 'muskmelon', 'apple',
        'orange', 'papaya', 'coconut', 'cotton', 'jute', 'coffee'],
        dtype=object)
```

Uses LabelEncoder to convert crop labels to numerical values.

```
In [37]: le=LabelEncoder()
df['label']=le.fit_transform(df['label'])
df
```

Out[37]:

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	20
1	85	58	41	21.770462	80.319644	7.038096	226.655537	20
2	60	55	44	23.004459	82.320763	7.840207	263.964248	20
3	74	35	40	26.491096	80.158363	6.980401	242.864034	20
4	78	42	42	20.130175	81.604873	7.628473	262.717340	20
...	...	...	...	...	...	...	...	...
2195	107	34	32	26.774637	66.413269	6.780064	177.774507	5
2196	99	15	27	27.417112	56.636362	6.086922	127.924610	5
2197	118	33	30	24.131797	67.225123	6.362608	173.322839	5
2198	117	32	34	26.272418	52.127394	6.758793	127.175293	5
2199	104	18	30	23.603016	60.396475	6.779833	140.937041	5

2200 rows × 8 columns

Prints the unique numerical values in the 'label' column.

In [38]: `df['label'].unique()`

Out[38]: `array([20, 11, 3, 9, 18, 13, 14, 2, 10, 19, 1, 12, 7, 21, 15, 0, 16, 17, 4, 6, 8, 5])`

Defines X (features) and y (target) based on the dataset.

In [39]: `X = df.iloc[:, :-1]`
`y = df["label"]`

`print("Shape of X:", X.shape)`
`print("Shape of y:", y.shape)`

Shape of X: (2200, 7)
 Shape of y: (2200,)

Splits the dataset into training and testing sets, and applies Min-Max scaling to normalize the feature values.

In [40]: `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)`
`scaler = MinMaxScaler()`
`X_train = scaler.fit_transform(X_train)`
`X_test = scaler.transform(X_test)`

`print("Shape of X train:", X_train.shape)`
`print("Shape of y train:", y_train.shape)`
`print("Shape of X test:", X_test.shape)`
`print("Shape of y test:", y_test.shape)`

Shape of X train: (1540, 7)
 Shape of y train: (1540,)
 Shape of X test: (660, 7)
 Shape of y test: (660,)

Modeling Classification algorithms

Defines and fits machine learning models (KNeighborsClassifier, SVC, GaussianNB, DecisionTreeClassifier, RandomForestClassifier, LogisticRegression) to the training data.

```
In [41]: knn=KNeighborsClassifier()
sv=SVC()
nb=GaussianNB()
dt=DecisionTreeClassifier(random_state=1,criterion= 'gini', max_depth= 9)
rf=RandomForestClassifier(random_state=1)
lr=LogisticRegression(C=100)

models=[knn,sv,nb,dt,rf,lr]
for model in models:
    print(""*25,model,""*25)
    model.fit(X_train,y_train)
    y_pred=model.predict(X_test)
    print(classification_report(y_pred,y_test))
```

***** KNeighborsClassifier() *****

	precision	recall	f1-score	support
0	1.00	1.00	1.00	31
1	1.00	1.00	1.00	28
2	1.00	0.91	0.95	32
3	1.00	1.00	1.00	28
4	1.00	1.00	1.00	25
5	1.00	1.00	1.00	28
6	1.00	0.97	0.99	37
7	1.00	1.00	1.00	44
8	0.97	0.89	0.93	35
9	1.00	1.00	1.00	33
10	0.96	0.90	0.93	29
11	0.96	1.00	0.98	27
12	1.00	0.97	0.98	33
13	0.85	1.00	0.92	28
14	1.00	1.00	1.00	33
15	1.00	1.00	1.00	36
16	1.00	1.00	1.00	23
17	1.00	0.95	0.98	21
18	0.97	1.00	0.98	28
19	1.00	1.00	1.00	31
20	0.84	1.00	0.91	21
21	1.00	1.00	1.00	29
accuracy			0.98	660
macro avg	0.98	0.98	0.98	660
weighted avg	0.98	0.98	0.98	660

***** SVC() *****

	precision	recall	f1-score	support
0	1.00	1.00	1.00	31
1	1.00	1.00	1.00	28
2	0.97	0.97	0.97	29
3	1.00	1.00	1.00	28
4	1.00	1.00	1.00	25
5	1.00	1.00	1.00	28
6	1.00	1.00	1.00	36
7	1.00	1.00	1.00	44
8	0.97	0.89	0.93	35
9	1.00	1.00	1.00	33
10	0.96	0.93	0.95	28
11	1.00	1.00	1.00	28
12	1.00	1.00	1.00	32
13	0.97	1.00	0.98	32
14	1.00	1.00	1.00	33
15	1.00	1.00	1.00	36
16	1.00	1.00	1.00	23
17	1.00	0.95	0.98	21
18	1.00	1.00	1.00	29
19	1.00	1.00	1.00	31
20	0.84	1.00	0.91	21
21	1.00	1.00	1.00	29
accuracy			0.99	660
macro avg	0.99	0.99	0.99	660
weighted avg	0.99	0.99	0.99	660

***** GaussianNB() *****

	precision	recall	f1-score	support
0	1.00	1.00	1.00	31

1	1.00	1.00	1.00	28
2	1.00	1.00	1.00	29
3	1.00	1.00	1.00	28
4	1.00	1.00	1.00	25
5	1.00	1.00	1.00	28
6	1.00	1.00	1.00	36
7	1.00	1.00	1.00	44
8	1.00	0.91	0.96	35
9	1.00	1.00	1.00	33
10	1.00	1.00	1.00	27
11	1.00	1.00	1.00	28
12	1.00	1.00	1.00	32
13	1.00	1.00	1.00	33
14	1.00	1.00	1.00	33
15	1.00	1.00	1.00	36
16	1.00	1.00	1.00	23
17	1.00	1.00	1.00	20
18	1.00	1.00	1.00	29
19	1.00	1.00	1.00	31
20	0.88	1.00	0.94	22
21	1.00	1.00	1.00	29
accuracy			1.00	660
macro avg	0.99	1.00	1.00	660
weighted avg	1.00	1.00	1.00	660

***** DecisionTreeClassifier(max_depth=9, random_state=1) *****

	precision	recall	f1-score	support
0	1.00	1.00	1.00	31
1	1.00	1.00	1.00	28
2	1.00	0.97	0.98	30
3	1.00	1.00	1.00	28
4	1.00	1.00	1.00	25
5	1.00	1.00	1.00	28
6	1.00	1.00	1.00	36
7	1.00	1.00	1.00	44
8	1.00	0.94	0.97	34
9	1.00	1.00	1.00	33
10	1.00	1.00	1.00	27
11	1.00	1.00	1.00	28
12	1.00	1.00	1.00	32
13	0.97	1.00	0.98	32
14	1.00	1.00	1.00	33
15	1.00	1.00	1.00	36
16	1.00	1.00	1.00	23
17	1.00	1.00	1.00	20
18	1.00	1.00	1.00	29
19	1.00	1.00	1.00	31
20	0.92	1.00	0.96	23
21	1.00	1.00	1.00	29
accuracy			1.00	660
macro avg	0.99	1.00	1.00	660
weighted avg	1.00	1.00	1.00	660

***** RandomForestClassifier(random_state=1) *****

	precision	recall	f1-score	support
0	1.00	1.00	1.00	31
1	1.00	1.00	1.00	28
2	1.00	1.00	1.00	29

3	1.00	1.00	1.00	28
4	1.00	1.00	1.00	25
5	1.00	1.00	1.00	28
6	1.00	1.00	1.00	36
7	1.00	1.00	1.00	44
8	1.00	0.94	0.97	34
9	1.00	1.00	1.00	33
10	1.00	1.00	1.00	27
11	1.00	1.00	1.00	28
12	1.00	1.00	1.00	32
13	1.00	1.00	1.00	33
14	1.00	1.00	1.00	33
15	1.00	1.00	1.00	36
16	1.00	1.00	1.00	23
17	1.00	1.00	1.00	20
18	1.00	1.00	1.00	29
19	1.00	1.00	1.00	31
20	0.92	1.00	0.96	23
21	1.00	1.00	1.00	29
accuracy				1.00 660
macro avg		1.00	1.00	1.00 660
weighted avg		1.00	1.00	1.00 660

***** LogisticRegression(C=100) *****				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	31
1	1.00	1.00	1.00	28
2	0.97	0.88	0.92	32
3	1.00	1.00	1.00	28
4	1.00	1.00	1.00	25
5	1.00	1.00	1.00	28
6	1.00	1.00	1.00	36
7	1.00	1.00	1.00	44
8	0.91	0.91	0.91	32
9	1.00	0.97	0.99	34
10	0.96	0.96	0.96	27
11	0.96	1.00	0.98	27
12	1.00	1.00	1.00	32
13	0.97	1.00	0.98	32
14	1.00	1.00	1.00	33
15	1.00	1.00	1.00	36
16	1.00	1.00	1.00	23
17	0.85	0.94	0.89	18
18	0.93	1.00	0.96	27
19	1.00	1.00	1.00	31
20	0.88	0.81	0.85	27
21	1.00	1.00	1.00	29
accuracy				0.98 660
macro avg		0.97	0.98	0.97 660
weighted avg		0.98	0.98	0.98 660

**From the Above Accuracies, We have adopt
RandomForestClassifier,GaussianNB,DecisionTreeClassifier as it has Maximum
Accuracy**

Accuracy comparison

Evaluates and compares the accuracy of different machine learning models on the test set.

```
In [42]: acc = [model.score(X_test, y_test) for model in models]
df1 = pd.DataFrame({'Algorithm': [model.__class__.__name__ for model in models], 'Accuracy': acc})
df1['Accuracy'] = df1['Accuracy'].round(2)
df1 = df1.sort_values(by='Accuracy', ascending=False)
df1
```

```
Out[42]:
```

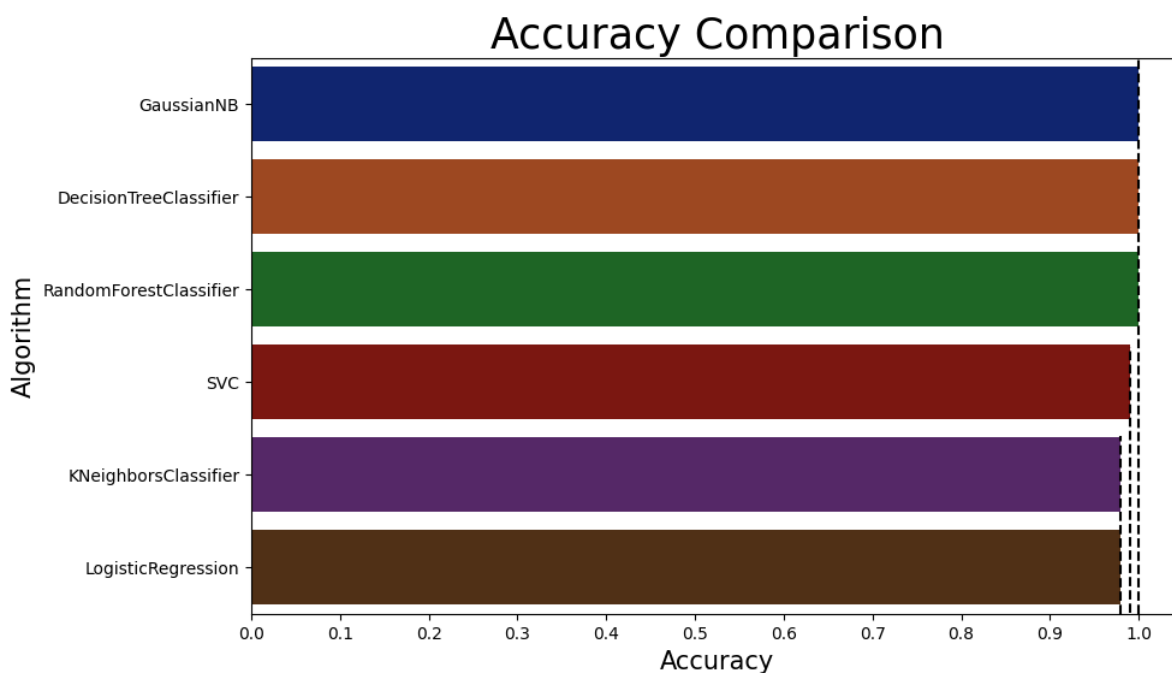
	Algorithm	Accuracy
2	GaussianNB	1.00
3	DecisionTreeClassifier	1.00
4	RandomForestClassifier	1.00
1	SVC	0.99
0	KNeighborsClassifier	0.98
5	LogisticRegression	0.98

Plots a bar chart comparing the accuracy of different models, with vertical lines highlighting specific accuracy values.

```
In [43]: plt.figure(figsize=(10, 6))
plt.title('Accuracy Comparison', fontsize = 25)
plt.xlabel('Accuracy', fontsize = 15)
plt.ylabel('Algorithm', fontsize = 15)
sns.barplot(x='Accuracy', y='Algorithm', data=df1, palette='dark')

plt.axvline(x=1.0, color='black', linestyle='--')
plt.axvline(x=0.99, color='black', linestyle='--', ymin=0.001, ymax=0.48)
plt.axvline(x=0.98, color='black', linestyle='--', ymin=0.0, ymax=0.32)

plt.xticks([i/10 for i in range(11)])
plt.savefig('plot.png', dpi=300, bbox_inches='tight')
plt.show()
```



Making a prediction

Uses selected models (Naive Bayes, Decision Tree, RandomForest) to make predictions for new climatic conditions and suggests a crop based on the predictions.

```
In [45]: model_pred=[nb,dt,rf]
for model in model_pred:
    print(""*14,model,""*14)
    y_new = np.array([[20, 30, 10, 15, 90, 7.5, 100]])
    new_pred = scaler.transform(y_new)
    new_pred1 = model.predict(new_pred)
    prediction = le.inverse_transform(new_pred1)
    print("The Suggested Crop for Given Climatic Condition is:", prediction)
    print()

***** GaussianNB() *****
The Suggested Crop for Given Climatic Condition is: ['orange']

***** DecisionTreeClassifier(max_depth=9, random_state=1) *****
The Suggested Crop for Given Climatic Condition is: ['orange']

***** RandomForestClassifier(random_state=1) *****
The Suggested Crop for Given Climatic Condition is: ['orange']
```

Our Predictive Model works very well. This Predictive model is usefull for Modern Farming